📊 Data Service

> 🚀 **Built for:** Real-time sauna monitoring and data analytics
> 🔧 **Tech Stack:** GraphQL, AWS Lambda, AWS IAM, Amazon Cognito
> 📊 **Data Sources:** AWS Timestream, InfluxDB, Amazon S3

🏷️ Enums

📊 SamplingMode

Defines how measurement data should be sampled:

Value	Description	Use Case
`NONE`	📈 No sampling, return all data points	Real-time monitoring
`SAMPLING`	⏱️ Sample data points at regular intervals	Performan optimizatic
`AVERAGE`	📊 Calculate average values for intervals	Data analysis

🗄️ DatabaseType

Specifies which database to use for queries:

On this page

- 🏷️ Enums
  - 📊 SamplingMode
  - 🗄️ DatabaseType
- 🔵 GraphQL
  - 🔍 Queries
    - 📋 `devicesMeasurementsList`
    - 📄 `devicesMeasurementsPdfGenerate`
    - ⚡ `devicesMeasurementsLatest`
    - 🎯 `devicesSessionsList`
    - 🏢 `organizationsSessionsList`
  - 💻 HTTP Request Examples
    - 🟧 Using JavaScript/fetch
    - 🐍 Using Python/requests
    - 🔧 Using cURL
  - 📡 Subscriptions
    - 🔧 **Setup Requirements**
    - 🔔 `devicesMeasurementsUpdateFeed`
    - 📊 Subscription Examples
    - 🟧 Using JavaScript/Node.js
  - 📋 Types
    - 🔄 MeasurementUpdate
    - 📮 MeasurementUpdateNotice
    - 🔥 Session
    - 📊 SessionItems
    - 📈 MeasurementItem
    - 📋 MeasurementItems
    - 📄 PdfReportLink
  - 📋 Sample Responses
    - ✅ Successful Query Response
    - ❌ Error Response
    - 📄 Pagination

Value	Description	Best For
`timestream`	⚡ AWS Timestream database	High-frequenc IoT data
`influxdb`	📈 InfluxDB database	Time-series analytics

🔵 GraphQL

The Data Service GraphQL API is the full API surface for accessing device data. The REST API documented below is a subset of these capabilities for common operations.

Note: All GraphQL requests require a Cognito ID token in the `Authorization: Bearer <idToken>` header. See the API Overview section for authentication setup.

Base URL: Get the GraphQL endpoint from the endpoints configuration: `endpoints.GraphQL.data.https`

🔍 Queries

📋 `devicesMeasurementsList`

> Retrieves a paginated list of measurements for a specific device within a time range.

Arguments:

Parameter	Type	Re
`deviceId`	`String!`	✅

startTimestamp	String!	✅
endTimestamp	String!	✅
samplingMode	SamplingMode	
sampleAmount	Int	
db	DatabaseType	
nextToken	String	

Returns: `MeasurementItems` - List of measurements with pagination support

Example:

```
query GetDeviceMeasurements {
  devicesMeasurementsList(
    deviceId: "DEVICE-ABC123-XYZ789"
    startTimestamp: "1735689600000"
    endTimestamp: "1735776000000"
    samplingMode: AVERAGE
    sampleAmount: 100
  ) {
    measurementItems {
      deviceId
      subId
      timestamp
      sessionId
      type
      data
    }
    nextToken
  }
}
```

---

devicesMeasurementsPdfGenerate

> Generates a PDF report of device measurements.

Arguments:

Parameter	Type	Re
deviceId	String!	✅
startTimestamp	String!	✅
endTimestamp	String!	✅
samplingMode	SamplingMode	
sampleAmount	Int	
db	DatabaseType	

Returns: `PdfReportLink` - URL to the generated PDF report

Example:

```
query GeneratePdfReport {
  devicesMeasurementsPdfGenerate(
    deviceId: "DEVICE-ABC123-XYZ789"
    startTimestamp: "1735689600000"
    endTimestamp: "1735776000000"
    samplingMode: AVERAGE
    sampleAmount: 1000
  ) {
    url
  }
}
```

---

devicesMeasurementsLatest

> Retrieves the most recent measurements for a device.

Arguments:

Arguments:

Parameter	Type	Required
deviceId	String!	✅

Returns: [MeasurementItem] - Array of latest measurements

Example:

```
query GetLatestMeasurements {
  devicesMeasurementsLatest(
    deviceId: "DEVICE-ABC123-XYZ789"
  ) {
    deviceId
    subId
    timestamp
    sessionId
    type
    data
  }
}
```

🎯 devicesSessionsList

> Retrieves a list of sessions for a specific device.

Arguments:

Parameter	Type	Re
deviceId	String!	✅
startTimestamp	AWSDateTime!	✅
endTimestamp	AWSDateTime!	✅
nextToken	String	⚪

Returns: SessionItems - List of sessions with pagination support

with pagination support

Example:

```
query GetDeviceSessions {
  devicesSessionsList(
    deviceId: "DEVICE-ABC123-XYZ789"
    startTimestamp: "2025-01-01T00:00
    endTimestamp: "2025-01-02T00:00:0
  ) {
    sessions {
      deviceId
      sessionId
      organizationId
      subId
      timestamp
      type
      durationMs
      stats
    }
    nextToken
  }
}
```

🏢 organizationsSessionsList

> Retrieves a list of sessions for all devices in an organization.

Arguments:

Parameter	Type	Re
organizationId	String!	✅
startTimestamp	AWSDateTime!	✅
endTimestamp	AWSDateTime!	✅
nextToken	String	🟣

Returns: `SessionItems` - List of sessions with pagination support

Example:

Example:

```
query GetOrganizationSessions {
  organizationsSessionsList(
    organizationId: "ORG-PROD-001"
    startTimestamp: "2025-01-01T00:00
    endTimestamp: "2025-01-02T00:00:0
  ) {
    sessions {
      deviceId
      sessionId
      organizationId
      subId
      timestamp
      type
      durationMs
      stats
    }
    nextToken
  }
}
```

💻 HTTP Request Examples

The following examples show how to make GraphQL queries using HTTP. Each example includes the complete authentication flow. For detailed authentication setup, token refresh, and error handling, see the API Overview section.

🟧 Using JavaScript/fetch

```
// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiGenerics = endpoints.Res
const graphqlEndpoint = endpoints.Gra

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

const query = `
  query GetDeviceMeasurements {
    devicesMeasurementsLatest(
      deviceId: "DEVICE-ABC123-XYZ789
```

```
    ) {
      deviceId
      subId
      timestamp
      sessionId
      type
      data
    }
  }
`;

const graphqlResponse = await fetch(g
  method: 'POST',
  headers: {
    'Content-Type': 'application/jso
    'Authorization': `Bearer ${idToke
  },
  body: JSON.stringify({ query })
});

const data = await graphqlResponse.js
console.log(data);
```

Using Python/requests

```
import requests

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_generics = endpoints["RestAp
graphql_endpoint = endpoints["GraphQL

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

query = """
query GetDeviceMeasurements {
  devicesMeasurementsLatest(
    deviceId: "DEVICE-ABC123-XYZ789"
  ) {
    deviceId
    subId
    timestamp
    sessionId
    type
    data
  }
}
"""

response = requests.post(
    graphql_endpoint,
```

```
    headers={
        'Content-Type': 'application/
        'Authorization': f"Bearer {id
    },
    json={'query': query}
)

data = response.json()
print(data)
```

🔧 Using cURL

```
# Get endpoints and authenticate
ENDPOINTS=$(curl -sS "https://api.har
REST_API_GENERICS=$(echo "$ENDPOINTS"
GRAPHQL=$(echo "$ENDPOINTS" | jq -r '

TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_GENERICS/auth/to
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j

QUERY='{"query":"query GetDeviceMeasu

curl -sS -H "Content-Type: applicatio
  -H "Authorization: Bearer $ID_TOKEN
  -X POST "$GRAPHQL" \
  --data "$QUERY" | jq '.'
```

📡 Subscriptions

> ⚠️ **Important**: Subscriptions are more complex than queries/mutations as they require WebSocket connections.

🔧 Setup Requirements

JavaScript/Node.js:

```
npm install aws-appsync aws-appsync-a
```

Note: `aws-appsync` requires `graphql` version 14.x or 15.0.0–15.3.0 (not 16+). If you encounter compatibility issues, install with: `npm install aws-appsync aws-appsync-auth-link graphql@14`

```
graphql-tag
```

Python:

```
pip install requests websocket-client
```

Note: Use the Cognito IdToken obtained via the REST API (see API Overview). Get endpoints from the Endpoints API; the client URL comes from `endpoints.GraphQL.data.https`. The `receiver` must be the JWT claim `cognito:username` from your IdToken.

🔔 `devicesMeasurementsUpdateFeed`

> Real-time feed of measurement updates.

Arguments:

Parameter	Type	Required	De
`receiver`	`ID!`	✅	The the

Returns: `MeasurementUpdateNotice` - Real-time updates of measurements

Example:

```
subscription SubscribeToMeasurements
  devicesMeasurementsUpdateFeed(recei
    receiver
    item {
      deviceId
      subId
      timestamp
      sessionId
      type
      data
    }
  }
}
```

📊 Subscription Examples

The following examples show complete subscription setup including authentication. For detailed authentication setup and token management, see the API Overview section.

Using JavaScript/Node.js

```
import { AWSAppSyncClient } from "aws
import { AUTH_TYPE } from "aws-appsyn
import gql from "graphql-tag";

// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiGenerics = endpoints.Res
const graphqlEndpoint = endpoints.Gra

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

// Extract user ID from JWT token (re
const tokenPayload = JSON.parse(Buffe
const userId = tokenPayload['cognito:

// Create AppSync client
const client = new AWSAppSyncClient({
  url: graphqlEndpoint,
  region: "eu-central-1",
  auth: {
    type: AUTH_TYPE.AWS_LAMBDA,
    token: async () => `Bearer ${idTo
  },
  disableOffline: true,
});

// Subscribe to measurements feed
const subscription = client.subscribe
  query: gql`
    subscription MeasurementsFeed {
      devicesMeasurementsUpdateFeed(r
        receiver
        item {
          deviceId
          subId
          timestamp
          sessionId
          type
          data
        }
      }
```

```
      }
    `
});

subscription.subscribe({
  next: (data) => console.log("Receiv
  error: (error) => console.error("Er
});
```

Using Python/requests

```
import json
import base64
import websocket
import requests
from uuid import uuid4

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_generics = endpoints["RestAp
graphql_endpoint = endpoints["GraphQL

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

def header_encode(header_obj):
    """Encode header using Base 64"""
    return base64.b64encode(json.dump

# Extract user ID from JWT token
token_payload = json.loads(base64.b64
user_id = token_payload['cognito:user

# Build WebSocket URL and host
wss_url = graphql_endpoint.replace('h
host = graphql_endpoint.replace('http

# Generate subscription ID
sub_id = str(uuid4())

# Create JWT authentication header
auth_header = {
    'host': host,
    'Authorization': f"Bearer {id_tok
}

# GraphQL subscription
gql_subscription = json.dumps({
    'query': f'subscription Measureme
    'variables': {}
})
```

```
# WebSocket event callbacks
def on_message(ws, message):
    message_object = json.loads(messa
    message_type = message_object['ty

    if message_type == 'connection_ac
        # Register subscription
        register = {
            'id': sub_id,
            'payload': {
                'data': gql_subscript
                'extensions': {'autho
            },
            'type': 'start'
        }
        ws.send(json.dumps(register))

    elif message_type == 'start_ack':
        print("✅ Subscription regis

    elif message_type == 'data':
        print("✅ Received subscripti
        # Stop subscription
        ws.send(json.dumps({'type': '

def on_open(ws):
    ws.send(json.dumps({'type': 'conn

# Create WebSocket connection
connection_url = wss_url + '?header='
ws = websocket.WebSocketApp(
    connection_url,
    subprotocols=['graphql-ws'],
    on_open=on_open,
    on_message=on_message
)

# Run WebSocket (use proper threading
ws.run_forever()
```

📋 Types

🔄 MeasurementUpdate

> Input type for measurement updates

Field	Type	Required	
deviceId	String!	✅	-
subId	String!	✅	-

timestamp	String!	✅	
sessionId	String		
type	String		
data	AWSJSON		

🎴 MeasurementUpdateNotice

> Measurement update notifications

Field	Type	Requ
receiver	ID!	✅
item	MeasurementItem!	✅

🔥 Session

> Sauna session representation

Field	Type	Re
deviceId	String!	✅
sessionId	String!	✅
organizationId	String!	✅
subId	String!	✅

timestamp	AWSDateTime!	✅
type	String	
durationMs	Float	
stats	AWSJSON	

📊 SessionItems

Paginated session results

Field	Type	Required
sessions	[Session!]!	✅
nextToken	String	

📈 MeasurementItem

Single measurement

Field	Type	Required	I
deviceId	String!	✅	
subId	String!	✅	
timestamp	String!	✅	
sessionId	String		

type	String		
data	AWSJSON		

📋 MeasurementItems

Paginated measurement results

Field	Type
measurementItems	[MeasurementItem!
nextToken	String

📄 PdfReportLink

PDF report link

Field	Type	Required	Descri
url	String		URL to genera PDF re

📋 Sample Responses

✅ Successful Query Response

```
{
  "data": {
    "devicesMeasurementsList": {
      "measurementItems": [
        {
          "deviceId": "DEVICE-ABC123-
          "subId": "C1",
          "timestamp": "1735689600000
          "sessionId": "session-456",
          "type": "temperature",
          "data": "{\"temp\": 80, \"h
        }
      ],
      "nextToken": "eyJsYXN0RXZhbHVhd
    }
  }
```

```
}
```

❌ Error Response

```
{
  "errors": [
    {
      "message": "Device not found",
      "locations": [{"line": 3, "colu
      "path": ["devicesMeasurementsLi
      "extensions": {
        "code": "DEVICE_NOT_FOUND",
        "exception": {
          "stacktrace": ["Error: Devi
        }
      }
    }
  ],
  "data": null
}
```

📄 Pagination

For paginated results, use the `nextToken` from the response in subsequent requests:

```
query GetMoreMeasurements {
  devicesMeasurementsList(
    deviceId: "DEVICE-ABC123-XYZ789"
    startTimestamp: "1735689600000"
    endTimestamp: "1735776000000"
    nextToken: "eyJsYXN0RXZhbHVhdGVkS
  ) {
    measurementItems {
      deviceId
      subId
      timestamp
      data
    }
    nextToken
  }
}
```

🌎 Configuration Reference: Harvia Endpoints API

📝 Note: Always fetch the latest configuration to ensure you're using the current endpoints, regions, and

client IDs. The configuration may change over time.

🌐 REST API

The REST API provides a subset of the full Data Service GraphQL API for common integration flows.

Note: All REST API endpoints require a Cognito ID token in the `Authorization: Bearer <idToken>` header. See the API Overview section for authentication setup.

Base URL: Get the REST API base URL from the endpoints configuration: `endpoints.RestApi.data.https`

📊 `GET /data/latest-data`

> Retrieves the most recent measurements for a specific device and cabin.

Arguments:

Parameter	Type	Required	D
`deviceId`	`string`	✅	A T
`cabinId`	`string`	🟣	S s i (M e v
			F

`cabinName`	`string`		

Notes:

- If neither `cabinId` nor `cabinName` is provided, `cabinId` defaults to `C1`.

Success Response (200):

```
{
  "deviceId": "ABC123",
  "shadowName": "C1",
  "subId": "C1",
  "timestamp": "2025-01-01T00:00:00.0
  "sessionId": "session-123",
  "type": "HEATER",
  "organization": "org-1",
  "data": {
    "temperature": 65.4,
    "humidity": 32.1
  }
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

📈 `GET /data/telemetry-history`

> Retrieves a paginated list of measurements for a specific device and cabin within a time range.

Arguments:

Parameter	Type	Require

deviceId	string	✅
cabinId	string	⚪
cabinName	string	⚪
startTimestamp	string	✅
endTimestamp	string	✅
samplingMode	string	⚪
sampleAmount	integer	⚪
nextToken	string	⚪

Notes:

Notes:

- If neither `cabinId` nor `cabinName` is provided, `cabinId` defaults to `C1`.
- Omit `samplingMode` to retrieve raw measurements.

Success Response (200):

```
{
  "deviceId": "ABC123",
  "shadowName": "C1",
  "measurements": [
    {
      "deviceId": "ABC123",
      "subId": "C1",
      "timestamp": "1700000000000",
      "organizationId": "org-1",
      "deviceCanSee": ["org-1"],
      "data": {
        "temperature": 64.2,
        "humidity": 31.4
      },
      "sessionId": "session-123",
      "type": "HEATER"
    }
  ],
  "nextToken": "eyJwYWdlIjoiMiJ9"
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

`GET /data/sessions`

> Retrieves a paginated list of sessions for a specific device and cabin within a time range.

Arguments:

Parameter	Type	Required
`deviceId`	`string`	✅

`cabinId`	`string`	🔮
`cabinName`	`string`	🔮
`startTimestamp`	`string`	✅
`endTimestamp`	`string`	✅
`nextToken`	`string`	🔮

Notes:

- If neither `cabinId` nor `cabinName` is provided, `cabinId` defaults to `C1`.
- Session statistics are returned in the `stats` object when available.

Success Response (200):

```
{
  "deviceId": "ABC123",
  "shadowName": "C1",
  "sessions": [
    {
```

```
      "deviceId": "ABC123",
      "subId": "C1",
      "sessionId": "session-123",
      "timestamp": "2025-01-01T00:00:
      "type": "HEATER",
      "durationMs": 5400000,
      "stats": {
        "avgTemperature": 63.1,
        "maxTemperature": 78.4,
        "measurementCount": 180
      }
    }
  ],
  "nextToken": "eyJwYWdlIjoiMiJ9"
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

💡 REST API Examples

Each example below shows the complete authentication flow. For detailed authentication setup, token refresh, and error handling, see the API Overview section.

🟧 Using JavaScript/fetch

```
// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiBase = endpoints.RestApi
const restApiGenerics = endpoints.Res

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

async function call(method, path) {
  const res = await fetch(`${restApiB
    method,
    headers: { Authorization: `Bearer
  });
```

```
  });
  if (!res.ok) throw new Error(`${res
  return res.json();
}

// Fetch latest data for device ABC12
const latestData = await call("GET",
console.log(latestData);

// Fetch telemetry history for the sa
const start = new Date("2025-01-01T00
const end = new Date("2025-01-02T00:0
const telemetryHistory = await call(
  "GET",
  `/data/telemetry-history?deviceId=A
);
console.log(telemetryHistory);

// Fetch sessions for the same device
const sessions = await call(
  "GET",
  `/data/sessions?deviceId=ABC123&cab
);
console.log(sessions);
```

Using Python/requests

```
import requests

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_base = endpoints["RestApi"][
rest_api_generics = endpoints["RestAp

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

def call(method, path):
    res = requests.request(method, f"
    if not res.ok:
        raise Exception(f"{res.status
    return res.json()

# Fetch latest data for device ABC123
latest_data = call("GET", "/data/late
print(latest_data)

# Fetch telemetry history for the sam
start = "2025-01-01T00:00:00.000Z"
end = "2025-01-02T00:00:00.000Z"
telemetry_history = call(
    "GET",
```

```
    f"/data/telemetry-history?deviceI
)
print(telemetry_history)

# Fetch sessions for the same device
sessions = call(
    "GET",
    f"/data/sessions?deviceId=ABC1238
)
print(sessions)
```

🔧 Using cURL

```
# Get endpoints and authenticate
ENDPOINTS=$(curl -sS "https://api.har
REST_API_BASE=$(echo "$ENDPOINTS" | j
REST_API_GENERICS=$(echo "$ENDPOINTS"

TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_GENERICS/auth/to
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j

# Fetch latest data for device ABC123
curl -sS -H "Authorization: Bearer $I
     "$REST_API_BASE/data/latest-data

START_ISO="2025-01-01T00:00:00.000Z"
END_ISO="2025-01-02T00:00:00.000Z"

# Fetch telemetry history for the sam
curl -sS -H "Authorization: Bearer $I
     "$REST_API_BASE/data/telemetry-h

# Fetch sessions for the same device
curl -sS -H "Authorization: Bearer $I
     "$REST_API_BASE/data/sessions?de
```
